SOFTWARE REQUIREMENTS SPECIFICATION

for

ROOM STATUS TRACKER
A Live Classroom Status Web Application

Version 1.0 | 9 October 2026

Prepared by
Sarah Singh (1024240071)
Prasuk Jain (1024240007)
Mansi Singh (1024240090)
Harshil Bansal (1024240040)
Group No.: 3X15

B.Tech Third Year, AI/ML
Faculty Advisor: Mr. Raghav B. Venkataramaiyer

Department of Computer Science and Engineering
Thapar Institute of Engineering and Technology, Patiala

Contents

1 Introduction

1.1 Purpose

1.2 Intended Audience and Reading Suggestions

1.3 Project Scope

2 Overall Description

2.1 Product Perspective

2.2 User Classes and Characteristics

2.3 Product Functions

2.4 Operating Environment

2.5 Design

3 System Features

3.1 Description and Priority

3.2 Functional Requirements

4 Other Nonfunctional Requirements

4.1 Performance Requirements

4.2 Security Requirements

4.3 Software Quality Attributes

4.4 Business Rules

5 Other Requirements

5.1 Implementation, Deployment, and Acceptance

References

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the requirements for Room Status Tracker, a web application intended to show the latest known status of classrooms, Lecture Theatres (LTs), and shared Activity Spaces. It describes the system scope, users, environment, functions, functional requirements, non-functional requirements, and business rules. It is intended to guide implementation, testing, and evaluation.

The current milestone has a React frontend with dashboard screens, room representations, status indicators, navigation, and role-oriented interface elements. The backend, database, authentication service, and live-data integration are planned for the next phase. Requirements describing these components apply to the intended complete system and do not imply that they are already implemented.

1.2 Intended Audience and Reading Suggestions

This document is intended for the student development team, faculty advisor, maintainers, testers, and representative users: students, Class Representatives (CRs), Society Representatives, teachers, and the Department Office of Administration (DOA). Readers seeking an overview should begin with Sections 1 and 2. Developers should focus on Sections 2 and 3. Testers should use the requirements in Sections 3 and 4 to derive test cases.

1.3 Project Scope

Room Status Tracker will provide one web-based place to check the latest reported status of CSED classrooms, Lecture Theatres, and shared Activity Spaces. It addresses the gap between a timetable and actual room use, such as when a class is cancelled, shifted, delayed, a teacher has not arrived, or a shared space is occupied.

The system will display room details, status, notes where available, and the time of the latest update. Students primarily view information. CRs and Society Representatives update rooms assigned to them. Teachers manage CR access for their classes and may perform authorized overrides. Admin/DOA has wider management access. Status changes will be associated with a user and timestamp and retained as history.

The planned architecture uses React, FastAPI/Python, PostgreSQL, SQLAlchemy, Alembic, college-email OTP or magic-link authentication, and an email service such as SendGrid or Resend. Hosting is planned for a frontend platform such as Vercel/Netlify and backend platform such as Render/Railway. Fully implemented authentication, persistent database operations, and live updates are outside the current frontend-only milestone and are planned for the next phase. Notifications, analytics, bulk assignment, and mobile/PWA support are possible future enhancements.

2 Overall Description

2.1 Product Perspective

Room Status Tracker complements the college timetable by presenting the latest reported room conditions. The timetable describes planned activity; this system is intended to describe the latest reported status. It follows a client-server architecture: the React frontend communicates with a FastAPI backend over HTTPS using JSON requests and responses. The backend applies business rules and access checks and uses PostgreSQL for persistent storage. An email service is planned for passwordless login verification.

The frontend foundation has been developed. Backend APIs, database integration, authentication, and live data integration remain planned components and must be evaluated separately from completed interface work.

2.2 User Classes and Characteristics

The system defines five user classes:

- Student: views room status, room details, notes, and last-update time. Read-only access to room status is the baseline policy.
- Class Representative (CR): views room information and updates only classrooms assigned to them.
- Society Representative: views room information and updates only Activity Spaces assigned to them.
- Teacher: views room information, manages CR access for classes within their responsibility, and performs suitable status overrides where authorized.
- Administrator / Department Office of Administration (Admin/DOA): manages room records and assignments, oversees access, and views status history.

Access is role-based and room-assignment-based. The backend must enforce permissions; hiding or showing frontend controls alone is not sufficient security.

2.3 Product Functions

The principal functions are:

- Room discovery and viewing, grouped by building or room type.
- Status presentation with a consistent indicator, note, and latest update time.
- Room-status updates by authorized users for assigned rooms.
- Role-specific navigation and available actions.
- College-email OTP or magic-link authentication (planned).
- Room and assignment administration by authorized teachers and Admin/DOA.
- Timestamped status history identifying the actor and change.
- Persistent storage and backend API integration (planned).

2.4 Operating Environment

The product is a browser-based web application intended for current versions of Google Chrome, Microsoft Edge, Mozilla Firefox, and Safari on Windows, macOS, and Linux. Responsive layouts should support common tablet and mobile screen sizes. Internet or campus-network connectivity is required to access the hosted application and communicate with the backend.

Planned technologies are React, FastAPI/Python, PostgreSQL, SQLAlchemy, Alembic, college-email OTP/magic-link authentication, and an email delivery provider such as SendGrid or Resend. Deployment is planned using a frontend host such as Vercel/Netlify and a backend host such as Render/Railway.

2.5 Design

The interface is organized around a room-status dashboard and room cards. Rooms may be grouped by building or type. Status badges should be consistent and readable, and status meaning should be conveyed through text as well as colour. All users should see consistent room information; available actions depend on role and room assignment.

The planned data model contains:

- User: unique identifier, name, email, role, creation timestamp, active status.
- Room: unique identifier, room code, building, floor, capacity, room type, active status.
- RoomAssignment: identifier, user, room, granting user, active flag, creation timestamp.
- StatusLog: identifier, room, status, optional note, acting user, timestamp.

Status values in the project design are ONGOING, NOT ARRIVED, CANCELLED, CHANGED, FREE, and OCCUPIED. The backend will be authoritative for permissions and persistent state.

3 System Features

3.1 Description and Priority

Priority 1 — View room status: the core user function; information should be easy to locate and interpret.

Priority 2 — Update room status: essential to keep information useful; only authorized users may update assigned rooms.

Priority 3 — Role- and assignment-based access: essential security and data-integrity control.

Priority 4 — Authentication: planned college-email verification before protected actions.

Priority 5 — Room and assignment management: authorized users maintain room records and responsibility assignments.

Priority 6 — Status history: supports accountability by recording who changed a status and when.

Priority 7 — Responsive navigation and room grouping: improves usability across device sizes and room categories.

3.2 Functional Requirements

FR-01 Room listing: The system shall display available rooms, grouped or filterable by building and/or room type where those attributes are available.

FR-02 Room details: The system shall show a room identifier, room type, latest recorded status, and last-update time, plus a note when available.

FR-03 Status vocabulary: The system shall represent status using ONGOING, NOT ARRIVED, CANCELLED, CHANGED, FREE, and OCCUPIED.

FR-04 Status update: The system shall allow an authenticated and authorized user to submit a status change for a room they may manage, with an optional note.

FR-05 Assignment enforcement: The system shall reject a status update if the user is not assigned to the room or otherwise explicitly authorized.

FR-06 Role enforcement: Students shall have viewing access; CRs shall update assigned classrooms; Society Representatives shall update assigned Activity Spaces; teachers shall manage CR access and perform authorized overrides; Admin/DOA shall have wider room and assignment-management access.

FR-07 Authentication: The complete system shall support college-email OTP or magic-link verification before granting access to protected functionality.

FR-08 Room administration: The system shall allow authorized administrators to create, edit, deactivate, and review room records.

FR-09 Assignment administration: The system shall allow authorized users to create, review, modify, and deactivate user-to-room assignments according to their permissions.

FR-10 Status history: The system shall store each accepted status change with room, status, optional note, acting user, and timestamp.

FR-11 History viewing: The system shall allow authorized roles to view status history according to the permission policy.

FR-12 Persistence: The complete system shall store users, rooms, assignments, and status history in PostgreSQL so accepted changes remain after reload or a new session.

FR-13 Feedback and errors: The system shall provide clear success or failure feedback. Failed or unauthorized updates shall not be presented as successful.

FR-14 Data freshness: The dashboard shall show the latest status returned by the backend. The completed implementation shall define a suitable refresh or live-update mechanism.

FR-15 Navigation: The system shall provide role-appropriate navigation and allow users to return to the room dashboard.

FR-16 Audit identity: Each status-history entry shall identify the responsible user and the time recorded.

FR-17 Deactivated records: Deactivated rooms or assignments shall not be used for new updates, while relevant history is retained.

FR-18 Secure API access: The backend shall validate authorization for each protected operation independently of frontend controls.

4 Other Nonfunctional Requirements

4.1 Performance Requirements

NFR-P-01 The dashboard and primary room-status information should load within 3 seconds under normal network conditions and the agreed test data volume. This is a target to validate, not a measured result.

NFR-P-02 After backend integration, a normal status-update request should receive a success or failure response within 2 seconds under normal conditions, excluding external email-delivery delays.

NFR-P-03 The system should remain usable with the expected room inventory and concurrent users. A representative load profile and load test shall be defined before final acceptance.

NFR-P-04 No backend performance figures are claimed for the current frontend-only milestone. Targets must be measured after integration.

4.2 Security Requirements

NFR-S-01 Protected functionality shall require a verified user identity.

NFR-S-02 The backend shall enforce role- and room-assignment-based authorization for every protected operation.

NFR-S-03 Authentication tokens and sessions shall be handled securely; secrets shall not be embedded in frontend source code or committed to source control.

NFR-S-04 Deployment shall use HTTPS, and user-supplied values shall be validated before storing or displaying them.

NFR-S-05 Database access shall use controlled credentials and parameterized/ORM-based queries. Credentials shall be stored in deployment configuration, not source code.

NFR-S-06 The application shall not permit one user to impersonate another or reveal sensitive account information.

NFR-S-07 Unauthorized update attempts shall be rejected and shall not change room state.

NFR-S-08 Status history shall retain actor identity and timestamp; access to history shall follow the permission policy.

4.3 Software Quality Attributes

Usability: status, notes, and update time should be understandable at a glance; colour must not be the only status cue.

Reliability: accepted updates should persist in the complete system. Failed updates must not be presented as saved.

Maintainability: frontend, backend, and database responsibilities should remain separated; API contracts and permission rules should be documented.

Compatibility: the web interface should support current mainstream browsers and common desktop/mobile viewports.

Accessibility: controls should have meaningful labels, adequate contrast, keyboard accessibility, and visible focus states.

Testability: requirements should map to test cases, including positive and negative permission checks.

Scalability: additional rooms and users should be accommodated without redesigning the core workflow.

4.4 Business Rules

BR-01 Room information is available to users who can access the application, subject to institutional access policy.

BR-02 Students are viewers and cannot update room status in the baseline permission model.

BR-03 A CR may update only classrooms assigned to that CR.

BR-04 A Society Representative may update only Activity Spaces assigned to that representative.

BR-05 Teachers may manage CR access for their classes and perform overrides only where authorized.

BR-06 Admin/DOA has wider administrative permissions for room records, assignments, and history oversight.

BR-07 Every accepted status change shall create a timestamped history entry associated with the responsible user.

BR-08 The latest displayed status shall reflect the most recent accepted change known to the system.

BR-09 A role alone does not grant permission to update every room; room assignment or explicit authorization must also be checked.

BR-10 Deactivated rooms or assignments shall not be available for new updates, but relevant historical records should remain available.

5 Other Requirements

5.1 Implementation, Deployment, and Acceptance

Deployment and cloud hosting: the complete application is planned for a hosted frontend service such as Vercel or Netlify and a hosted backend service such as Render or Railway. PostgreSQL will provide persistent

storage. Deployment must support secure environment configuration and HTTPS. Cloud hosting is part of the intended architecture; actual hosting configuration is pending backend integration.

Data migration and maintenance: Alembic is planned for controlled database schema migrations. The team should maintain version control, document environment setup, and test migrations before applying them to production data.

Testing and acceptance: the completed system should undergo functional, integration, authorization, usability, and performance testing. Acceptance should include all user roles, assigned and unassigned room updates, correct status-history creation, authentication failure cases, and persistence after reload.

Known limitations and assumptions: the current milestone contains frontend screens and interface elements only. Backend APIs, database persistence, authentication, server-side authorization, and live data refresh are planned. This specification does not claim those components are already implemented. Push notifications, analytics, bulk assignment, and mobile/PWA support are optional future enhancements.

References:

1. React Documentation — https://react.dev/

2. FastAPI Documentation — https://fastapi.tiangolo.com/

3. PostgreSQL Documentation — https://www.postgresql.org/

4. SQLAlchemy Documentation — https://www.sqlalchemy.org/

5. Alembic Documentation — https://alembic.sqlalchemy.org/